Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026
Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

CareTree – Advanced Algorithm-Based Hospital Care Management System

<M. Ramya Sree > <2520030094 >
<S. Lohitha > <2520030109 >
<D. Teja Sadwika > <2520030536>

COURSE INSTRUCTOR

Dr. Swathi
Professor
Department of Computer Science and Engineering

CareTree – Advanced Algorithm-Based Hospital Care Management System

1. Introduction :

CareTree is a web-based Hospital Care Management System designed to simplify and organise hospital operations through a single platform. Hospitals handle a large amount of information every day, including patient details, doctor schedules, appointments, bed availability, emergency cases, medical inventory, pharmacy services, laboratory reports, and billing. Managing all these activities manually can be time-consuming and may lead to errors, delays, and difficulties in accessing information.

The CareTree project aims to provide a centralised system where hospital staff can manage essential activities efficiently. It helps organise patient and doctor records, schedule appointments, monitor bed and ward availability, manage emergency cases, track inventory, and maintain billing information. By bringing these functions together, the system supports better coordination between hospital departments.

This project also demonstrates the practical application of Advanced Algorithms in healthcare management. Algorithms for string matching, dynamic programming, network flow, approximation, and randomised computation can be applied to solve different hospital-related problems. For example, string-matching algorithms can help search for patient records, dynamic programming can support scheduling decisions, and network-flow algorithms can model the allocation of limited hospital resources.

CareTree is developed as an academic case study to demonstrate how software systems and algorithmic techniques can help make hospital operations more organised, accessible, and efficient.

2. Problem Statement :

In many hospitals, different departments handle separate tasks, such as patient registration, doctor appointments, bed allocation, pharmacy inventory, laboratory services, and billing. When these activities are managed through disconnected systems or manual records, staff may find it difficult to obtain updated information quickly.

For example, a receptionist may need to check multiple records to find an available appointment, while the ward staff may need to contact another department to confirm whether a bed is vacant. Similarly, searching for a patient using an incorrectly entered name can be difficult if the system supports only exact text matching.

These problems can result in delays, duplicate records, scheduling conflicts, and inefficient use of hospital resources. During emergencies, the need to identify available beds, doctors, and other resources becomes especially important.

Therefore, there is a need for a centralised hospital management system that organises important information, simplifies routine tasks, and supports efficient searching and resource allocation. CareTree addresses this need by integrating essential hospital functions into one web-based platform.

3. Objectives of the Project :

The main objectives of CareTree are:

1. **Patient Management :** Maintain patient details and make patient information easier to access.
2. **Doctor Management :** Organise doctor details, departments, and availability.
3. **Appointment Scheduling :** Help manage appointments and identify scheduling conflicts.

4. **Bed and Ward Management :** Display and update bed availability and occupancy status.
5. **Emergency Management :** Support the organisation of emergency cases and resource information.
6. **Inventory Management :** Maintain records of medical supplies and their availability.
7. **Pharmacy and Laboratory Management :** Organise medicine-related information and laboratory service records.
8. **Billing Management :** Maintain billing details and support the tracking of hospital charges.
9. **Algorithm Integration :** Demonstrate how advanced algorithms can be applied to searching, scheduling, and resource-allocation problems.

4. Application of Advanced Algorithms in CareTree :

An important feature of this case study is the connection between hospital operations and advanced algorithmic techniques. Different algorithms can be used to solve different types of problems.

A. String-Matching Algorithms :

String-matching algorithms are useful when searching for patient names, doctor names, medical records, or other text-based information. Algorithms such as Knuth–Morris–Pratt (KMP), the Z algorithm, and Rabin–Karp can search for patterns within text efficiently.

Example: Suppose a hospital has thousands of patient records and a staff member wants to find a patient whose name is “Ravi Kumar.” A search function can compare the entered name with stored records. Fuzzy matching can also help identify possible matches when the name is entered with a minor spelling mistake, such as “Ravi Kumr.” The system should verify the patient's identity before displaying or updating sensitive information.

B. Dynamic Programming :

Dynamic programming solves problems by breaking them into smaller subproblems and storing the results to avoid repeated calculations. It can be useful for certain scheduling and optimisation problems.

Example: Suppose a hospital needs to arrange appointments for several patients while considering doctor availability, appointment duration, and time-slot restrictions. A scheduling algorithm can evaluate possible arrangements and identify a suitable schedule according to defined rules. Dynamic programming may be appropriate when the scheduling problem has overlapping subproblems and a suitable recurrence relation.

C. Network-Flow Algorithms :

Network-flow algorithms are used to model how limited resources can be allocated through a network. Algorithms such as Ford–Fulkerson, Edmonds–Karp, and Dinic can calculate maximum flow in a network.

Example: Imagine a hospital receiving patients who require beds in different wards. The hospital has limited beds, and each patient may be eligible for only certain wards based on the care required. The allocation problem can be represented as a flow network, where patients and wards are connected according to eligibility and capacity. A maximum-flow algorithm can help determine a feasible allocation under the model's constraints. Actual clinical decisions must still be made by qualified healthcare professionals.

D. Approximation Algorithms:

Some optimisation problems are computationally difficult to solve exactly for large inputs. Approximation algorithms can produce useful solutions within defined performance limits.

Example: A hospital wants to select a limited number of monitoring stations to cover several important areas, such as wards, corridors, and emergency rooms. If one station can monitor multiple areas, an approximation algorithm may help select a smaller set of stations while covering all required locations. This can support planning, although real-world safety and coverage requirements must also be considered.

E. Randomised Algorithms :

Randomised algorithms use random choices during computation. They can be useful for efficient searching, hashing, and probabilistic testing.

Example: A hospital information system may use hashing to locate records efficiently. Randomised techniques can also be used in algorithmic experiments to compare performance across different inputs. However, randomised algorithms should not replace required security checks or clinical validation.

5. Practical Case Study Examples

The following scenarios illustrate how CareTree could be used in a hospital environment.

- **Case Study 1:** Patient Registration and Record Search

  A patient named Ananya visits the hospital for a consultation. The receptionist enters the patient's details into CareTree. If the patient has visited before, the receptionist searches for the existing record instead of creating a duplicate. String matching can help locate the record, while carefully designed fuzzy search can suggest possible matches if the name is entered incorrectly. The receptionist verifies the patient's identity before proceeding.

- **Case Study 2**: Appointment Scheduling

  A patient requests an appointment with a cardiologist. CareTree can display the doctor's available time slots and allow staff to select a suitable appointment. Before confirming the booking, the system checks whether the selected slot is already occupied. Scheduling logic can help reduce conflicts and improve the organisation of appointments.

- **Case Study 3**: Hospital Bed Allocation

  An emergency patient requires admission. The hospital staff check CareTree to identify beds marked as available. The system can display the available beds and relevant ward information. If several patients need beds simultaneously, resource-allocation algorithms can help analyse possible assignments according to bed capacity and eligibility rules. Clinical priority and treatment requirements remain under the control of healthcare professionals.

- **Case Study 4:** Medical Inventory Management

  A hospital pharmacy needs to monitor the availability of medicines and other medical supplies. CareTree can maintain inventory records and help staff identify items that are running low. For example, if the quantity of a particular medicine falls below a specified threshold, the system can flag it for review and restocking. This can reduce the risk of overlooking important supplies.

- **Case Study 5:** Billing and Department Coordination

  After a patient completes a consultation and laboratory tests, the hospital needs to organise the associated charges. CareTree can maintain billing records and bring together relevant service

information. This reduces the need to search through separate records and helps staff review the patient's bill more conveniently.

6. Expected Results and Benefits :

CareTree aims to provide a structured and user-friendly platform for managing essential hospital activities. Its expected benefits include:

- Centralised information: Patient, doctor, appointment, bed, inventory, and billing information can be organised in one system.
- Faster record searching: Search algorithms can improve the process of locating relevant records.
- Better appointment organisation: Availability checks can help reduce scheduling conflicts.
- Improved resource visibility: Bed and inventory status can be displayed for easier monitoring.
- Reduced manual effort: Organised digital records can reduce repetitive paperwork.
- Practical algorithm learning: The project demonstrates how advanced algorithms can be connected to real-world healthcare management problems.

The effectiveness of these benefits depends on the implementation, data quality, testing, and the accuracy of the system's records. They should be evaluated through suitable test cases rather than assumed without verification.

7. Conclusion :

CareTree – Advanced Algorithm-Based Hospital Care Management System demonstrates how a web-based application can help organise hospital operations through a centralised platform. By combining patient

management, doctor scheduling, appointment booking, bed and ward management, emergency support, inventory, pharmacy, laboratory services, and billing, the project addresses common information-management challenges in hospitals.

The project also connects advanced algorithmic concepts to practical healthcare scenarios. String matching supports record searching, dynamic programming can assist suitable scheduling problems, network flow can model resource allocation, approximation algorithms can support selected optimisation tasks, and randomised techniques can improve certain computational processes.

In the future, CareTree could be enhanced with a secure database, role-based access control, real-time updates, notifications, mobile support, and improved scheduling features. With proper testing, privacy protections, and clinical oversight, the system could serve as a useful foundation for exploring technology-assisted hospital management.

ALGORITHM / PSEUDO CODE :

1. Introduction :

CareTree is a web-based Hospital Care Management System designed to organise hospital activities such as patient registration, doctor management, appointment scheduling, bed allocation, emergency management, inventory management, and billing. The system follows a sequence of steps to accept user input, validate information, process requests, update records, and display the results.

The project demonstrates the application of advanced algorithmic concepts to healthcare management. String-matching algorithms can support patient searches, scheduling algorithms can help manage appointments, network-flow techniques can model bed allocation, and threshold-based checks can support inventory management.

The following pseudocode describes the overall workflow of CareTree and the major algorithms used in its modules. It is language-independent and can be implemented using JavaScript, Java, Python, or another suitable programming language.

2. Overall Algorithm of CareTree :

Step-by-step procedure :-

1. Start the system: Open the CareTree website and initialise the application.
2. Display the dashboard: Show the available modules, such as Patient Management, Doctor Management, Appointments, Bed Management, Inventory, and Billing.
3. Accept user selection: Allow the user to choose the required operation.
4. Validate input: Check whether the entered information is complete and follows the required format.

5. Process the request: Execute the algorithm associated with the selected module.
6. Update records: Save the changes to the application's data store, if the operation is valid.
7. Display the result: Show a success message, the requested information, or an appropriate error message.
8. Repeat the process: Allow the user to perform another operation until they choose to exit.
9. End the system: Close the session or return to the appropriate exit screen.

Pseudocode 1: Main CareTree Algorithm

```
ALGORITHM CareTree_Main :
BEGIN
   INITIALISE application
   LOAD hospital records
   DISPLAY "Welcome to CareTree"
   REPEAT
      DISPLAY Main Dashboard
      DISPLAY "1. Patient Management"
      DISPLAY "2. Doctor Management"
      DISPLAY "3. Appointment Scheduling"
      DISPLAY "4. Bed Management"
      DISPLAY "5. Inventory Management"
      DISPLAY "6. Billing Management"
      DISPLAY "7. Exit"
      INPUT choice
      IF choice = 1 THEN
         CALL Patient_Management
      ELSE IF choice = 2 THEN
```

```
        CALL Doctor_Management
    ELSE IF choice = 3 THEN
        CALL Appointment_Management
    ELSE IF choice = 4 THEN
        CALL Bed_Management
    ELSE IF choice = 5 THEN
        CALL Inventory_Management
    ELSE IF choice = 6 THEN
        CALL Billing_Management
    ELSE IF choice = 7 THEN
        DISPLAY "Exiting CareTree"
    ELSE
        DISPLAY "Invalid choice. Try again."
    END IF
  UNTIL choice = 7
  SAVE valid changes
  DISPLAY "CareTree session ended"
END
```

Explanation : The main algorithm controls the overall flow of the application. It displays the dashboard and directs each request to the appropriate module. The loop allows users to perform multiple operations in one session.

3. Patient Management Algorithm :

Patient Management is responsible for registering new patients, storing their details, and retrieving existing records. Each patient can have a unique patient ID, name, age, contact information, and other required details.

Before saving a new record, the system validates the input and checks whether the patient ID already exists. During searching, the system compares the entered search term with stored patient information.

Step-by-step procedure :-

1. Select Patient Management from the dashboard.
2. Choose Add Patient or Search Patient.
3. For a new patient, enter the required information.
4. Validate mandatory fields and the patient ID.
5. Check for duplicate IDs.
6. Store the record if the information is valid.
7. For a search request, compare the search term with existing records.
8. Display matching results or a message indicating that no match was found.

Pseudocode 2: Patient Registration and Search

```
ALGORITHM Patient_Management :
BEGIN
  DISPLAY "1. Add Patient"
  DISPLAY "2. Search Patient"
  INPUT operation
  IF operation = 1 THEN
    INPUT patientID, patientName, age, contact
    IF required fields are missing THEN
      DISPLAY "Please complete all required fields"
    ELSE IF patientID already exists THEN
      DISPLAY "Patient ID already exists"
    ELSE
      CREATE new patient record
      STORE patient record
      DISPLAY "Patient registered successfully"
    END IF
```

```
    ELSE IF operation = 2 THEN
        INPUT searchTerm
        found ← FALSE
        FOR each patient IN PatientRecords DO
            IF searchTerm matches patient name
               OR searchTerm matches patient ID THEN
                DISPLAY patient record
                found ← TRUE
            END IF
        END FOR
        IF found = FALSE THEN
            DISPLAY "No matching patient found"
        END IF
    ELSE
        DISPLAY "Invalid operation"
    END IF
END
```

Example : Suppose the patient database contains the names Ananya Rao, Ravi Kumar, and Priya Sharma. When a staff member searches for “Ravi Kumar,” the algorithm compares the search term with each record and displays the matching patient.

For larger datasets, an indexing method or an efficient string-searching algorithm can improve search performance. Fuzzy matching may also suggest possible matches when a name contains a spelling error, but the patient's identity must be verified before a record is accessed or modified.

4. Appointment Scheduling Algorithm :

The Appointment Management module helps staff schedule consultations based on doctor availability. It checks whether the selected doctor has an appointment at the requested date and time.

Step-by-step procedure :-

1. Enter the patient ID and doctor ID.
2. Select the appointment date and time.
3. Check whether the patient and doctor records exist.
4. Verify that the requested time is valid.
5. Search the appointment records for a conflict.
6. If the slot is available, create the appointment.
7. Otherwise, ask the user to select another time.
8. Display the result.

Pseudocode 3: Appointment Scheduling

```
ALGORITHM Schedule_Appointment :
BEGIN
   INPUT patientID, doctorID, date, time
   IF patientID does not exist THEN
      DISPLAY "Invalid patient ID"
      RETURN
   END IF
   IF doctorID does not exist THEN
      DISPLAY "Invalid doctor ID"
      RETURN
   END IF
   IF date or time is invalid THEN
      DISPLAY "Invalid date or time"
```

```
        RETURN
    END IF
    FOR each appointment IN Appointments DO
        IF appointment.doctorID = doctorID
            AND appointment.date = date
            AND appointment.time = time
            AND appointment is active THEN
            DISPLAY "Time slot unavailable"
            RETURN
        END IF
    END FOR
    CREATE new appointment
    STORE appointment
    DISPLAY "Appointment booked successfully"
END
```

Example : If Dr. Kumar already has an active appointment at 10:00 AM on a particular day, another appointment cannot be assigned to the same doctor at the same time under this simplified model. The user can choose a different available slot.

In a real implementation, appointment creation should use a database transaction or equivalent concurrency control so that two users cannot book the same slot simultaneously.

5. Bed Management and Allocation Algorithm :

The Bed Management module maintains the status of hospital beds, such as Available, Occupied, and Reserved. When a patient requires admission, the system checks for a suitable available bed.

Step-by-step procedure :-

1. Receive the patient's admission request.
2. Identify the required ward or bed category.
3. Retrieve the list of beds.
4. Check the status and eligibility of each bed.
5. Select a suitable available bed.
6. Update the bed status to Occupied or Reserved, according to the admission workflow.
7. Link the allocation to the patient record.
8. Display the allocation result.
9. If no suitable bed is available, display a message for staff review.

Pseudocode 4: Bed Allocation

```
ALGORITHM Allocate_Bed :
BEGIN
   INPUT patientID, requiredWard
   IF patientID does not exist THEN
      DISPLAY "Invalid patient ID"
      RETURN
   END IF
   selectedBed ← NONE
   FOR each bed IN BedRecords DO
      IF bed.ward = requiredWard
        AND bed.status = "Available"
        AND bed is suitable for the patient THEN
         selectedBed ← bed
         BREAK
      END IF
   END FOR
```

```
    IF selectedBed = NONE THEN
        DISPLAY "No suitable bed currently available"
        DISPLAY "Refer request to hospital staff"
    ELSE
        RESERVE selectedBed
        UPDATE selectedBed status
        LINK bed to patientID
        DISPLAY "Bed allocation completed"
    END IF
END
```

Example : A patient requires admission to a general ward. The algorithm examines the beds in that ward and selects the first available bed that meets the required conditions. If no suitable bed exists, the system informs the staff rather than assigning an occupied bed.

For larger resource-allocation problems, network-flow algorithms such as Edmonds–Karp or Dinic can be explored to model multiple patients, bed eligibility, and ward capacities. Actual admission and clinical-priority decisions must remain under authorised healthcare staff.

6. Inventory Management Algorithm :

The Inventory Management module tracks the quantity of medicines and hospital supplies. It helps identify items that need restocking.

Pseudocode 5: Inventory **Check**

```
ALGORITHM Check_Inventory :
BEGIN
    INPUT itemID, quantityToIssue
    IF itemID does not exist THEN
```

```
        DISPLAY "Item not found"
        RETURN
    END IF
    IF quantityToIssue <= 0 THEN
        DISPLAY "Invalid quantity"
        RETURN
    END IF
    IF quantityToIssue > currentStock(itemID) THEN
        DISPLAY "Insufficient stock"
        RETURN
    END IF
    currentStock(itemID) ←
        currentStock(itemID) - quantityToIssue
    SAVE updated stock
    IF currentStock(itemID) <= reorderLevel(itemID) THEN
        DISPLAY "Restocking required"
    ELSE
        DISPLAY "Stock updated successfully"
    END IF
END
```

Example : If a pharmacy has 100 units of a supply and issues 20 units, the remaining stock becomes 80. If the reorder level is 25 units, the system flags the item for restocking only when its quantity reaches that threshold or falls below it.

7. Complexity and Algorithm Analysis :

Algorithm analysis helps estimate how the time and memory requirements change as the amount of data increases.

Module	**Basic approach**	**Approximate time complexity**
Patient search	Linear search through n records	(O(n)
Appointment conflict check	Scan a appointments	O(a)
Bed allocation	Scan b beds	(O(b)
Inventory lookup	Linear search through items	O(m)

Here, n is the number of patient records, a is the number of appointments, b is the number of beds, and m is the number of inventory items. These complexities describe the simple pseudocode above. Using indexes, hash tables, or more advanced algorithms can improve performance for larger datasets.

8. Conclusion :

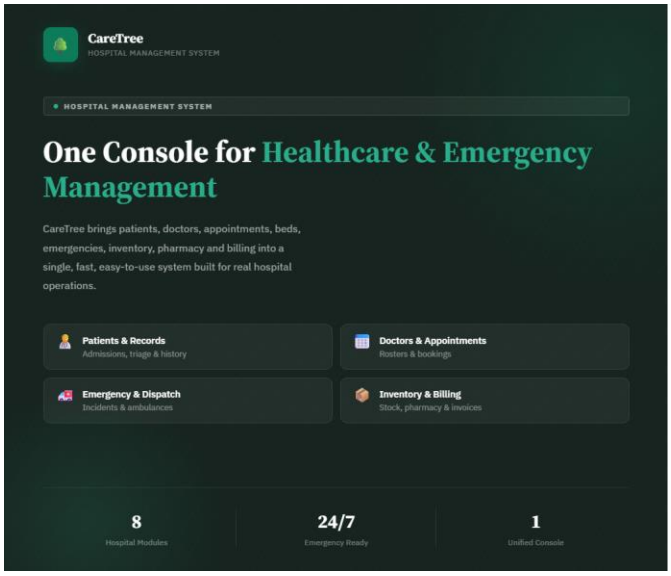
The pseudocode describes the major operations of CareTree, from opening the application to managing patient records, scheduling appointments, allocating beds, and updating inventory. Each module follows a structured sequence of input validation, processing, record updates, and output generation.

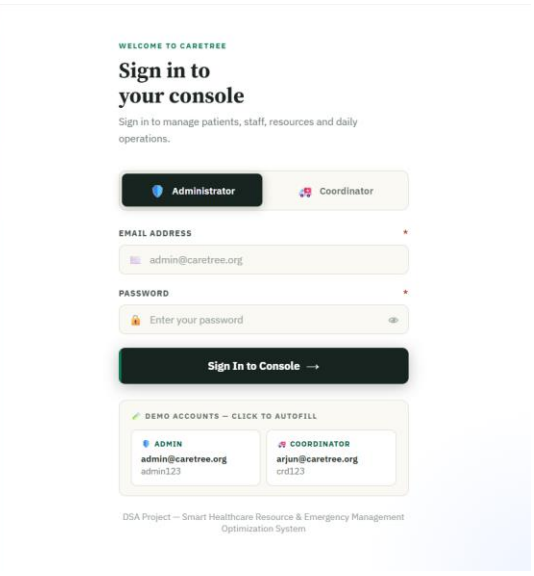
The project also illustrates how algorithm design supports hospital management. String matching can improve record searching, scheduling logic can prevent appointment conflicts, and resource-allocation algorithms can help analyse bed

availability. Inventory checks support stock monitoring, while complexity analysis helps evaluate performance as hospital data grows.

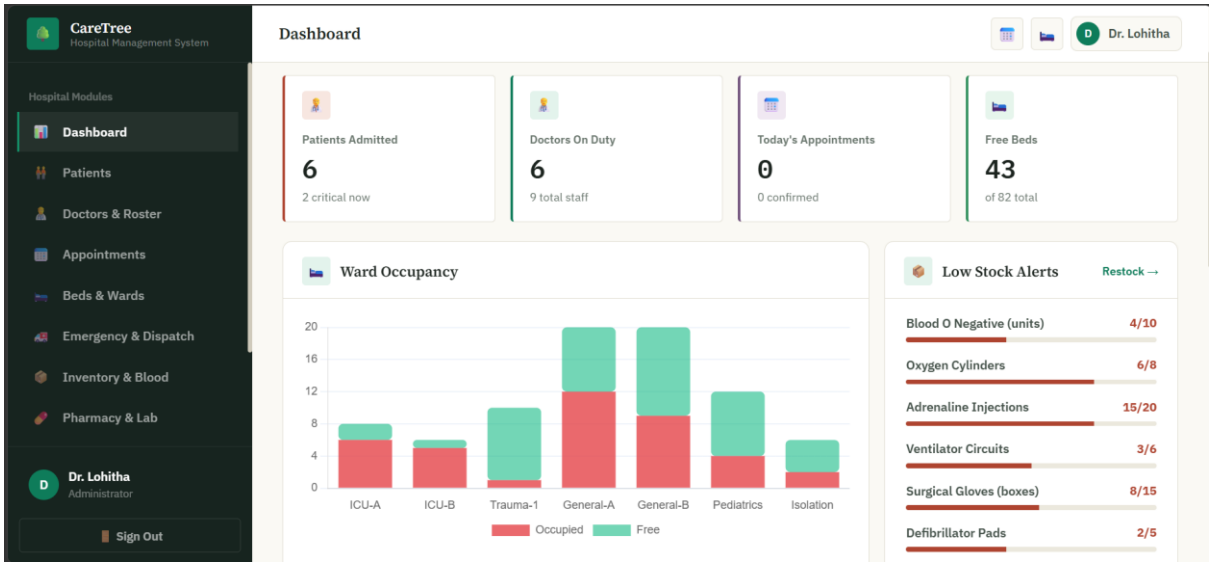
These algorithms provide a foundation for implementing CareTree. Their practical effectiveness depends on correct coding, reliable data storage, access control, testing, and appropriate validation by hospital staff.

RESULTS :





-



The CareTree – Advanced Algorithm-Based Hospital Care Management System provides a centralised interface for managing important hospital activities. The project includes modules for patient management, doctor management, appointment scheduling, bed management, emergency services, inventory, pharmacy, and billing.

The system is designed to make hospital information easier to access and organise. The following screenshots should be included in the project report to demonstrate the application interface and its functionality.